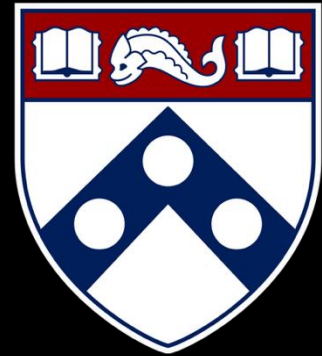


Secure Systems Engineering and Management



A Data-driven Approach



Secure by Design
& Intro to Threat Modeling

Michael Hicks

UPenn CIS 7000-003
Spring 2026

You Know the Attacks. Now What?

Scenario: You're hired to build the university's new course management system.

You've studied the CWE Top 25:

- Buffer overflows, SQL injection, XSS, path traversal, ...

Question: How do you make sure *none* of those end up in production?



(not real! Drawn by CoPilot)

Our Running Example: Limes

A university course management system (Canvas, Blackboard, ...)

- **Users:** Students, TAs, instructors, admins
- **Data:** Grades, submissions, personal records (FERPA)
- **Features:** File upload, discussion forums, quizzes, video integration
- **Integrations:** SSO, plagiarism detection, LTI plugins, email

We'll use this system throughout to ground every concept

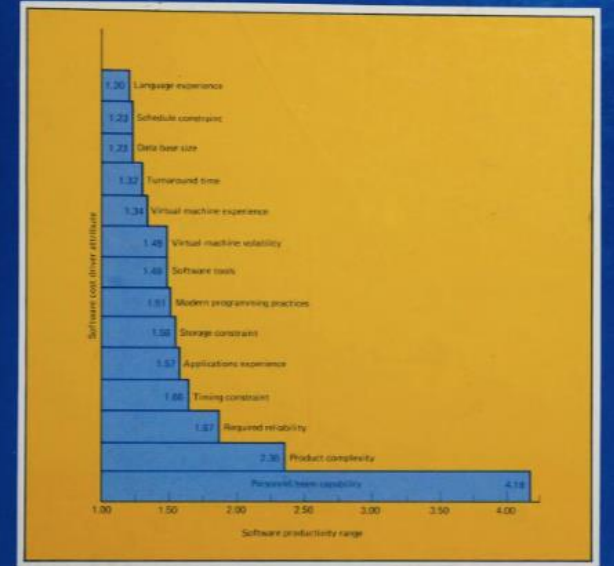
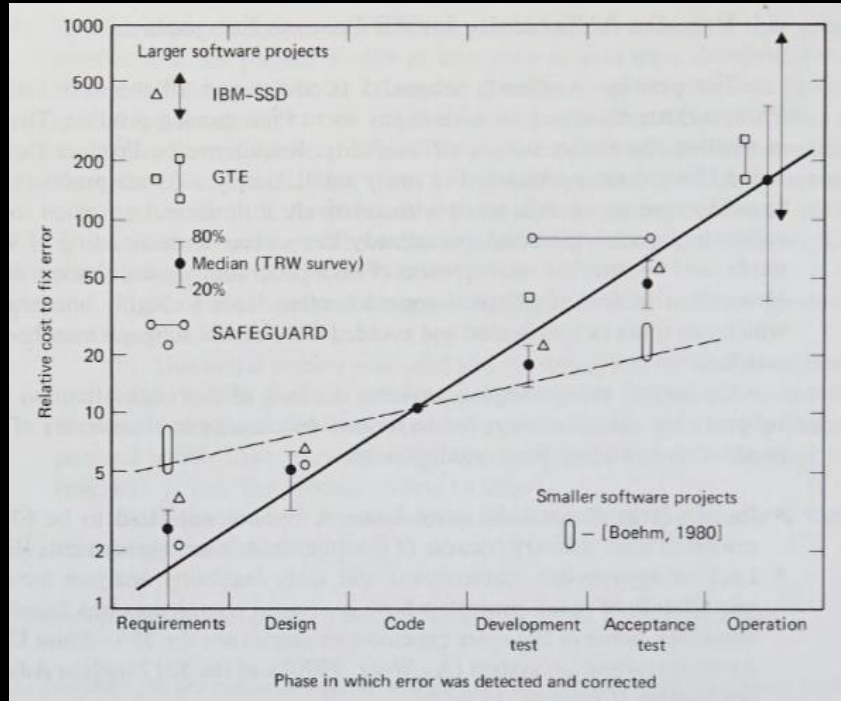
The Reactive Security Trap

Most of cybersecurity grew up **reacting** to insecure systems:

- Patch management
- Configuration hardening
- Malware detection
- Incident response
- ...

“Frequent security bulletins and advisories, disruptive patching, complex configuration management, malware analysis and detection, and vulnerability disclosure may have created more security ‘fatigue’ than progress.”
– CIS/SAFECode, *Secure by Design* (2025)

The Cost of Finding Bugs Late



**SOFTWARE
ENGINEERING
ECONOMICS**

BARRY W. BOEHM

Published 1981

What Does “Secure by Design” Mean?

Secure by Design = building security in from the start, as a first-class design constraint

Not a binary state. A continuum of risk management.

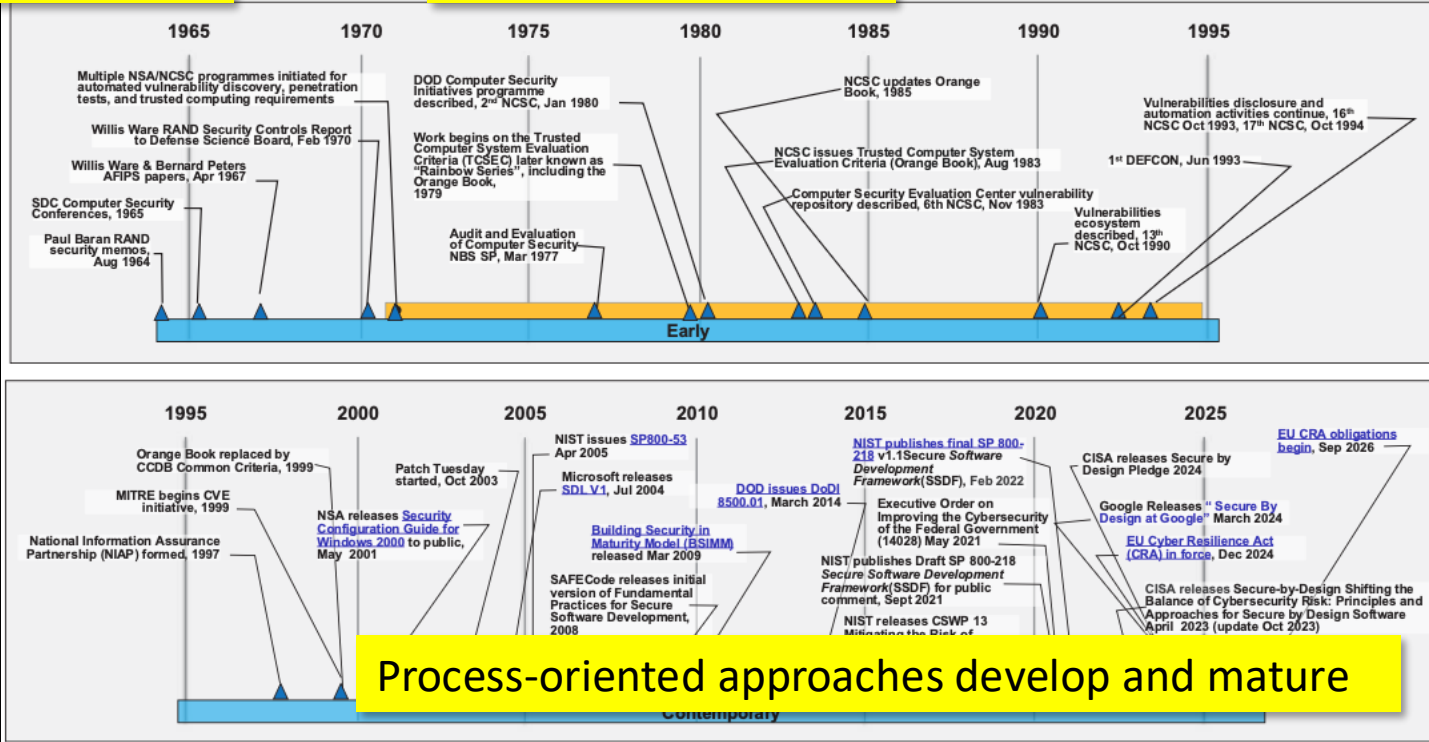
Three foundational truths:

1. **Perfect security is impossible.** The goal is to manage risk.
2. **Not all security practices are equal.** Prioritize.
3. **“Secure” is never finished.** It’s a continuous feedback loop.

The problem
is recognized

Government solution
attempt: certification

Internet: Boom



Microsoft was a target, then became a leader

← → ↻ 🔍 wired.com/2002/01/bill-gates-trustworthy-computing/ ☆ 🌐 📁 📧 👤

☰ **WIRED** 👤 NEWSLETTERS [SUBSCRIBE](#)

***THIS IS THE** e-mail Bill Gates sent to every full-time employee at Microsoft, in which he describes the company's new strategy emphasizing security in its products.*From: Bill Gates

Sent: Tuesday, January 15, 2002 5:22 PM

To: Microsoft and Subsidiaries: All FTE

Subject: Trustworthy computing

January 2002

Every few years I have sent out a memo talking about the highest priority for Microsoft. Two years ago, it was the kickoff of our .NET strategy. Before that, it was several memos about the importance of the Internet to our future and the ways we could make the Internet truly useful for people. Over the last year it has become clear that ensuring .NET is a platform for Trustworthy Computing is more important than any other part of our work. If we don't do this, people simply won't be willing -- or able -- to take advantage of all the other great work we do. Trustworthy Computing is the highest priority for all the work we are doing. We must lead the industry to a whole new level of Trustworthiness in computing.

Microsoft

BEST PRACTICES

THE SECURITY DEVELOPMENT LIFECYCLE

secure software
DEVELOPMENT SERIES

July 2006



SDL: A Process for Developing Demonstrably More Secure Software

Secure by Design

It's time to build cybersecurity into the design and manufacture of technology products.

2023

As America's cyber defense agency, CISA is charged with defending our nation against ever-evolving cyber threats and to understand, manage, and reduce risk to the cyber and physical infrastructure that Americans rely on every hour of every day. But, as we introduce more unsafe technology

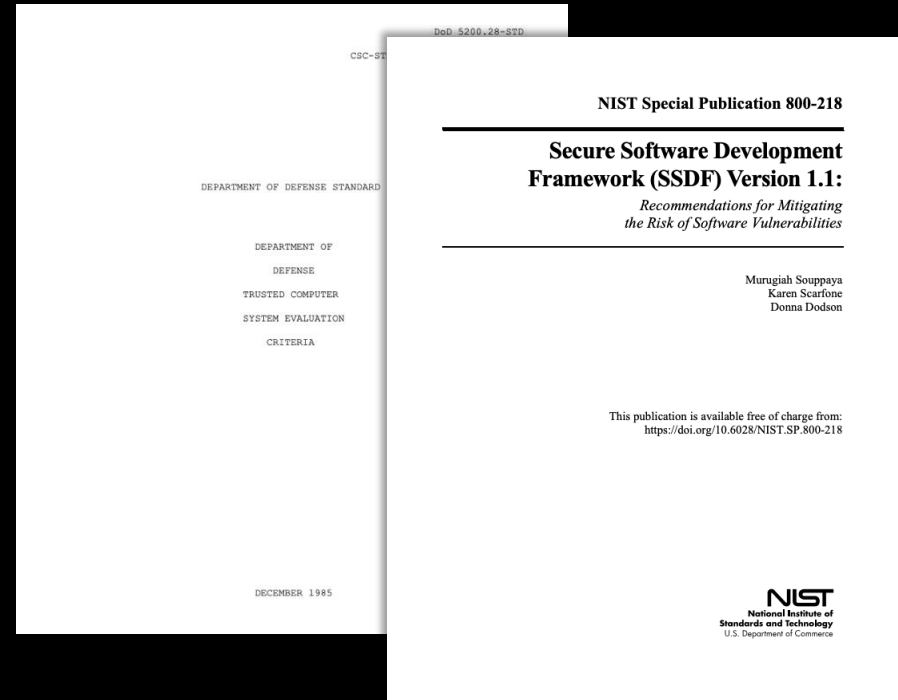
As a nation, we have allowed a system where the cyber security of our products is left to the discretion of consumers and small organizations and away from the federal government that increasingly run our digital lives. Americans need a system where consumers can trust the safety and integrity of the technology they use.

“Every technology provider must take ownership at the executive level to ensure their products are secure by design.”

“prioritize the security of customers as a core business requirement” and “implement Secure by Design principles to significantly decrease the number of exploitable flaws.”

Lessons from History

- Principles understood since the 1960s-70s
- Early approaches (Orange Book) were too rigid to scale
- The Internet era outpaced security processes
 - Microsoft pivoted to catch up
- Modern frameworks (SSDF, CIS Controls) aim for the **sweet spot**: practical, evaluable, adaptable



The CIS/SAFECode paper builds on the NIST SSDF to create **independent, practical, and actionable guidance** – bridging broad principles and concrete practices.

Six Considerations: Overview

1. Secure Software Design
2. Secure Development
3. Secure Default Configuration



4. Supply Chain Security
5. Code Integrity
6. Vulnerability Remediation

Consideration 1: Secure Software Design



Design begins with **security objectives**:

Requirement	Definition
Confidentiality	Protect information from unauthorized disclosure
Integrity	Protect information from unauthorized modification
Availability	Provide access when required
Accountability	Non-repudiation via permanent audit records

Secure Design: Security Features

These objectives are typically achieved through:



Feature	Intent
A uthentication	Reliably identifying users (passwords, MFA, SSO)
A uthorization	Granting access based on roles and rights
A uditing	Creating reliable, immutable records of actions

“The gold standard”

These must be **correctly implemented** and **impossible to bypass**.

DRAFT

date October 3, 1974

THE PROTECTION OF INFORMATION IN COMPUTER SYSTEMS

by

Jerome H. Saltzer and Michael D. Schroeder

Abstract--This tutorial paper explores the mechanics of protecting information stored in computer systems. It begins by discussing desired functions, design principles, and examples of elementary protection and user authentication mechanisms. The second major part examines in depth the principles of modern descriptor-based protection architectures and the relation between capability systems and access-control-list systems. This discussion ends with a brief analysis of protected subsystems and protected objects. The last part of the paper reviews the state of the art and current research problems and also provides suggestions for further reading.

The authors are with Project MAC and the Computer Science section of the Department of Electrical Engineering, Massachusetts Institute of Technology, Cambridge, Massachusetts, 02139

Secure Design: Saltzer & Schroeder's Principles

Classic design principles (1975) that remain essential:

- **Least privilege:** Grant minimum access needed for the task
- **Fail-safe defaults:** If in doubt, *deny* access
- **Complete mediation:** Check *every* access to *every* object
- **Economy of mechanism:** Keep security-critical code simple
- **Separation of privilege:** Require multiple conditions for access

Secure Design: The Confused Deputy

The **confused deputy problem**: a program with elevated privileges is tricked into misusing them on an attacker's behalf.

CMS example: The server-side PDF renderer runs with system privileges to read uploaded files. A malicious “assignment” triggers it to read `/etc/passwd` instead.

Confused deputies appear everywhere: CSRF, SSRF, insecure deserialization, ...

Secure Design Requires Threat Modeling

“Secure system design teams in industry frequently rely on a process called threat modeling to review their designs.”

– CIS/SAFECode, *Secure by Design*

- Analyze data flows to identify where an attacker might strike
- Identify countermeasures for each threat
- Supported by frameworks, tools, and training

We'll do this for our CMS in the second half of today's lecture.

Consideration 2: Secure Development



The process of writing, testing, and maintaining code securely.

A secure development process must include:

- A **“bug bar”**: which security bugs block release?
- **Coding standards**: memory-safe languages, approved libraries, safe patterns
- **Security tooling**: static analysis, dynamic analysis, dependency scanning
- **Code review** focused on security-critical paths
- **Security testing** beyond functional tests (regression and pen testing)
- **Developer training** on secure coding

The Secure Development Toolchain

Tool Class	What It Does	When
SAST (Static Analysis)	Scans source code for vulnerability patterns	During coding & CI
SCA (Software Composition Analysis)	Checks dependencies for known CVEs	Build time
DAST (Dynamic Analysis)	Tests running application for vulnerabilities	Testing/staging
Fuzzing	Feeds unexpected inputs to find crashes	Testing
Code Review	Human analysis of security-critical code	Before merge

Consideration 3: Secure Default Config



Ship it **locked down**. Users opt *out* of security, not *in*.

The 80% rule: If a feature, privilege, or interface is not needed by 80% of users, **disable it by default**.

CMS examples:

- New courses: all content **private** by default
- File uploads: restricted file types (.pdf, .docx – not .exe, .html)
- API access: **disabled** by default
- Admin panel: not accessible from public internet
- Session timeout: reasonable default (not “forever”)

Consideration 4: Supply Chain Security



Modern software is **mostly code you didn't write**.

Each dependency is a **trust decision**:

- Is it actively maintained?
- Has it been audited for security?
- What happens when a vulnerability is found in it?

Supply Chain: Log4Shell as Case Study

December 2021: CVE-2021-44228 in Apache Log4j

- A logging library used by thousands of Java applications
- Remote code execution via a simple log message
- Many organizations didn't know they were using it



Key lesson: You must know what's in your software (**SBOM**) so you can respond when a dependency is compromised.

Consideration 5: Code Integrity



Ensuring the software delivered to users is **what you intended to build**.

Threat: Malicious code inserted during development or delivery

Real-world attacks:

- **SolarWinds (2020):** Build system compromised; malicious update sent to 18,000 organizations
- **XZ Utils (2024):** Backdoor inserted by a trusted maintainer into a core Linux library
- **CodeCov (2021):** CI/CD tool compromised; credentials stolen from thousands of repos

Consideration 6: Vulnerability Remediation



No software ships bug-free. The question is: **what happens when bugs are found?**

Three phases:

1. **Immediate remediation:** Fix the reported vulnerability, ship a patch
2. **Variant analysis:** Search for *similar* vulnerabilities elsewhere in the codebase
3. **Process improvement:** Update tools, training, and practices to prevent recurrence

The Remediation Learning Loop

Every vulnerability is **data about your process**.

- Repeated XSS findings → rethink your templating approach
- Repeated authz bugs → redesign your authorization model
- Repeated dependency vulns → improve your vetting process

AI and Secure Software Development

Four intersections of AI/ML with software security:

#	Intersection	Implication
1	AI components <i>in</i> your software	Their security must be assured
2	AI tools used to <i>write</i> code	AI-generated code needs the same security scrutiny
3	AI tools to <i>find</i> vulnerabilities	Emerging aid for threat modeling and analysis
4	Adversaries using AI to <i>attack</i>	Offensive capabilities evolving rapidly

It's Not Just Technical – It's Organizational

“An organizational culture that prioritizes security must underlie implementation of engineering and operational practices for SbD.”
– CIS/SAFECode, *Secure by Design*

Security by Design requires:

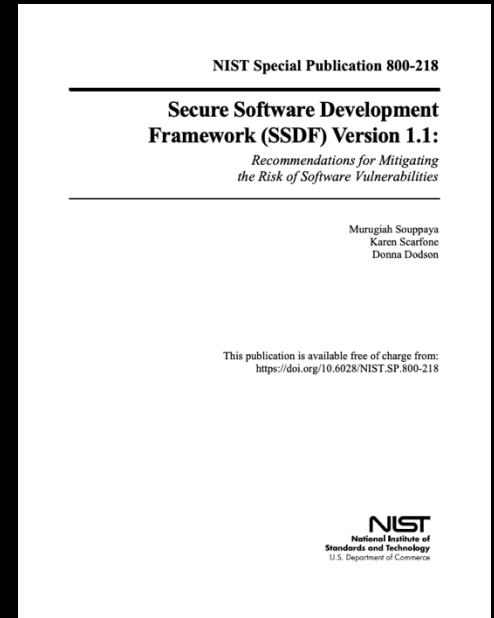
- **Management commitment** and budget
- **Defined roles** and responsibilities
- **Training** for all development staff
- **Security champions** embedded in teams
- **Artifacts** produced naturally by the process

Building Security In Maturity Model



The NIST SSDF: Four Practice Groups

Practice Group	Focus
Prepare the Organization (PO)	Define process, assign roles, train people, choose tools
Protect the Software (PS)	Access control on code, code signing, release integrity
Produce Well-Secured Software (PW)	Design, code, test, and configure securely
Respond to Vulnerabilities (RV)	Identify, assess, remediate, and learn



Assessment: How Do You Know It's Working?

“Secure by Design” doesn't mean “vulnerability-free.”

It means the organization has **addressed the six considerations** with evidence.

Evidence comes from **artifacts** naturally produced by development:

- Threat models and design documents
- Code review records and tool outputs
- Bug tracking history (including security bugs)
- Test results (including security tests)
- Vulnerability response records

“Evidence created solely to satisfy an assessor should be considered with great suspicion.”

How Does Your Experience Compare?

Think about a software project you've worked on:

- A class project, internship, open source contribution, personal project

Which of the six considerations did you address?

1. Secure Design
2. Secure Development
3. Secure Default Configuration
4. Supply Chain Security
5. Code Integrity
6. Vulnerability Remediation

Which did you completely ignore?



Putting It All Together

We've seen that secure software requires:

- **Secure Design:** Architecture that resists attack (CIA, access control, least privilege)
- **Secure Development:** Practices and tools throughout the coding process
- **Secure Defaults:** Ship locked down; minimize attack surface
- **Supply Chain Security:** Know and vet what you depend on
- **Code Integrity:** Protect the development and delivery pipeline
- **Vulnerability Remediation:** Find, fix, and learn from bugs continuously

All of this rests on **organizational commitment** and the SSDF framework.

The First Step: Understanding Your Threats

The foundation of secure design is **knowing what you're defending against**.

This requires a **systematic process** – not just intuition.

That process is called **threat modeling**.

Why Threat Modeling?

- Secure design requires **systematically** identifying threats
- Not just intuition or experience – a **structured, repeatable process**
- Asking “What could go wrong?” *before* writing code

“Threat modeling is a family of structured, repeatable processes that allow you to make rational decisions to secure applications, software, and systems.”

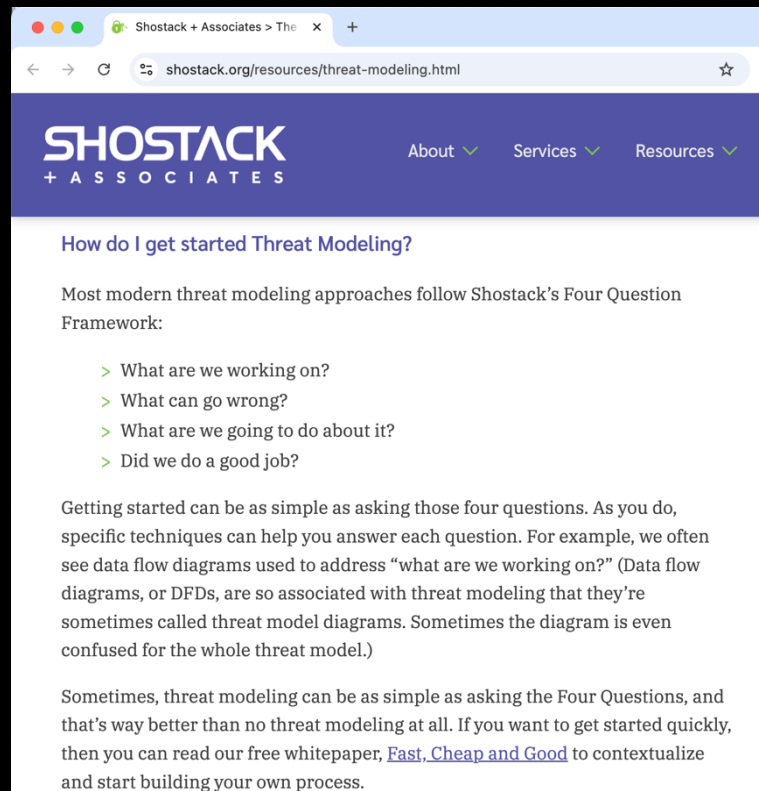
– Adam Shostack



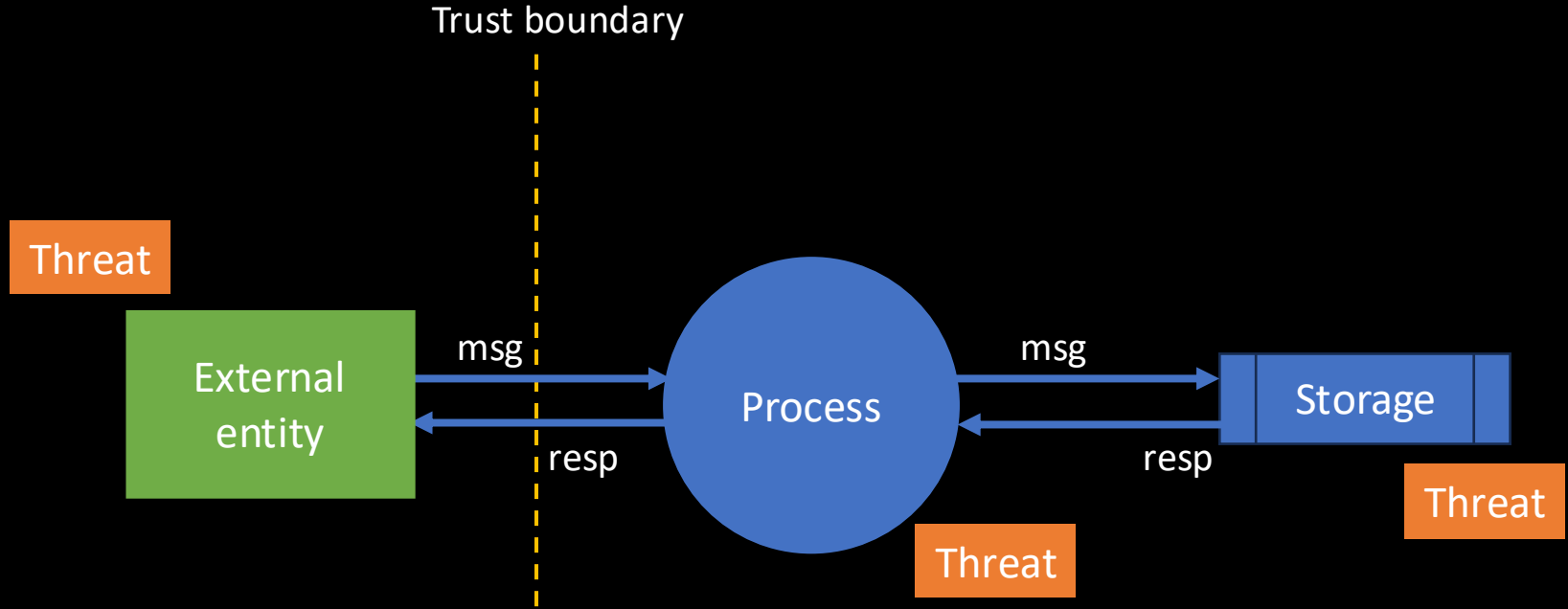
The Four Questions of Threat Modeling

Shostack's four-question framework

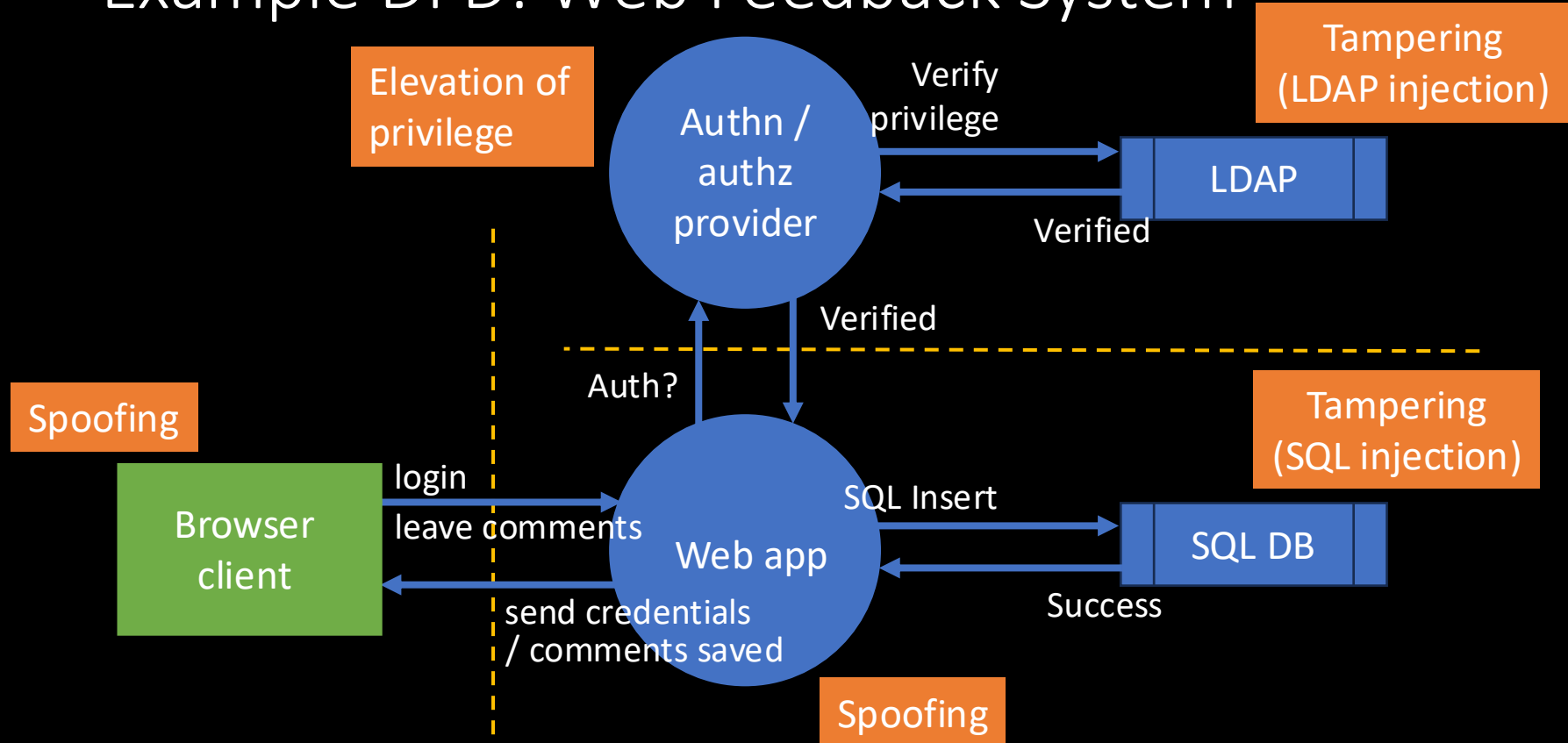
1. **What are we working on?** Describe the system
2. **What can go wrong?** Identify threats
3. **What are we going to do about it?** Mitigate, accept, transfer, eliminate
4. **Did we do a good job?** Validate the analysis



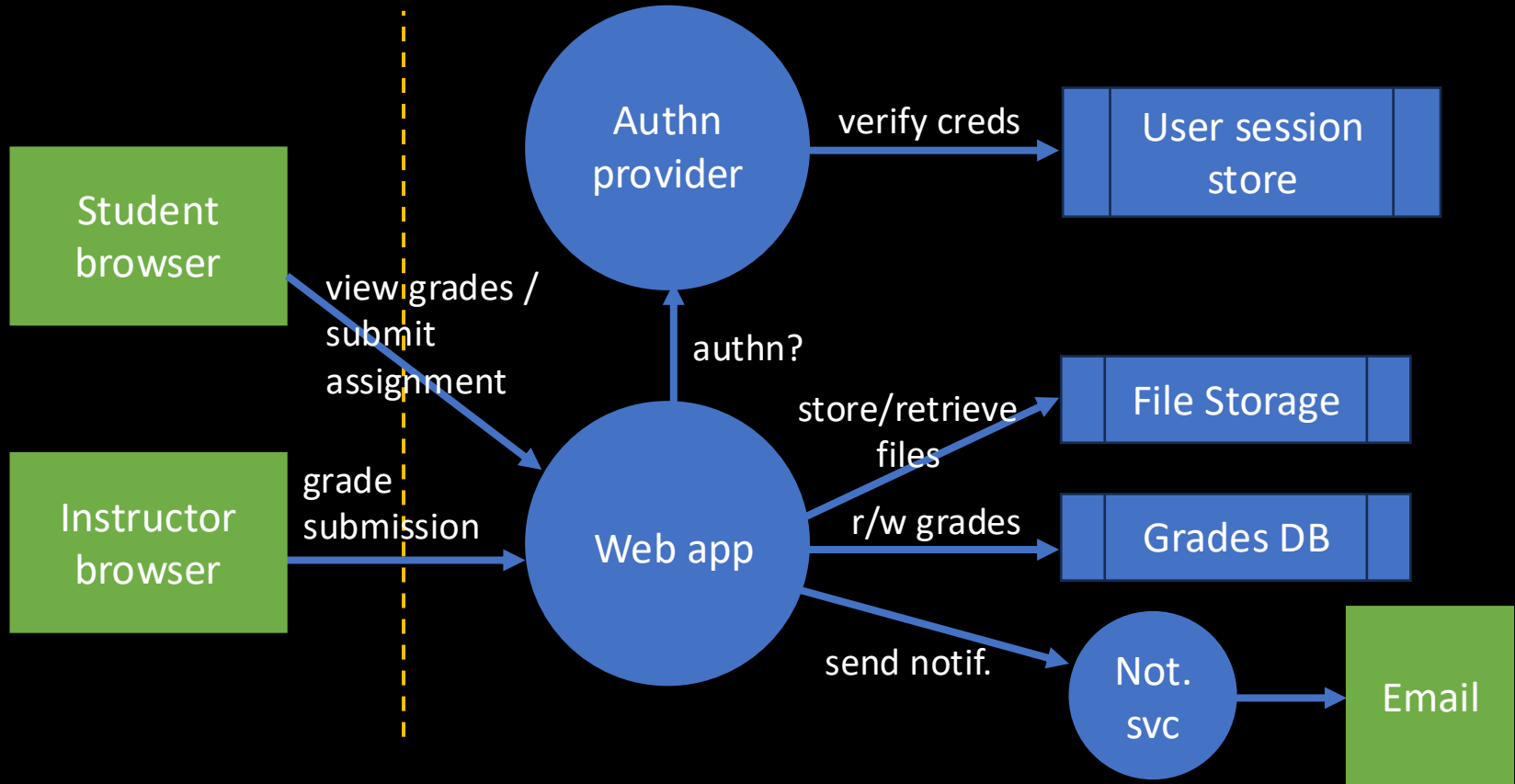
What are we working on? Data Flow Diagrams



Example DFD: Web Feedback System



DFD: CMS Assignment Submission



The threats to our products

April 1, 1999 — By Loren Kohnfelder and Praerit Garg

The growing use of computer systems to store data critical to businesses, as well as users' personal data, makes them very attractive targets for security attacks. Successful attacks can lead to loss of privacy, disclosure of sensitive data, and disruption or denial of service—losses that can cost millions of dollars. The Microsoft Security Task Force has defined a security threat model that it recommends all Microsoft product teams adopt to secure our products for our customers.

The **S.T.R.I.D.E.** security threat model should be used by all MS products to identify various types of threats the product is susceptible to during the design phase. Identifying the threats is the first step in a [proactive security analysis process](#). Threats are identified based on the design of the product. The next steps in the process are identifying the vulnerabilities in the implementation and then taking measures to close security gaps.

S.T.R.I.D.E. stands for:

- [Spoofing of user identity](#)
- [Tampering with data](#)
- [Repudiability](#)
- [Information disclosure \(privacy breach\)](#)
- [Denial of Service \(D.o.S.\)](#)
- [Elevation of privilege](#)

Some attacks can be very sophisticated and have several steps. In such attacks, one minor break-in leads to another, and eventually substantial system damage is done. In most such cases, one of the links is the weakest, and the security of the entire system typically is no better than its weakest link. Finding and improving such weak links is how threat analysis helps improve the security of our products and services.

This article describes various threat categories in the S.T.R.I.D.E. model and provides examples of vulnerabilities that may be exploited by various kinds of attacks to make a threat a reality. This is intended to help you identify potential vulnerabilities in your product during a security analysis.

Each threat is discussed in the context of Microsoft products, which fall into the following five categories:

- **Server operating systems** Windows NT/2000 Server
- **Client operating systems** Windows NT/2000 Workstation, Win9x, WinCE, Internet Explorer
- **Client/server applications** Exchange, SQL, etc.
- **Desktop applications** Office, etc.
- **Web and media applications** WebEssentials, portal Web sites, etc.

But first, let's define some important terms that we'll use throughout this piece and that have very precise meaning in security discussions.

STRIDE: A Systematic Way to Find Threats

Threat	Violates	Ask yourself...
Spoofing	Authentication	Can someone pretend to be another user or system?
Tampering	Integrity	Can data be modified without detection?
Repudiation	Non-repudiation	Can someone deny performing an action?
Information Disclosure	Confidentiality	Can unauthorized parties access data?
Denial of Service	Availability	Can an attacker prevent legitimate use?
Elevation of Privilege	Authorization	Can someone gain permissions they shouldn't have?

STRIDE Applied to CMS Example (generally)

Attack Class	Intent	CMS Example
Spoofing	Impersonate a user or system	Faking a login as another student
Tampering	Unauthorized modification	Changing a grade in transit
Repudiation	Deny having performed an action	Instructor denies changing a grade
Information Disclosure	Unauthorized access to data	Viewing another student's submissions
Denial of Service	Disrupt system operation	Overwhelming the system during finals
Elevation of Privilege	Gain unauthorized capabilities	Student gaining instructor access

STRIDE Applied: CMS Assignment Submission

Data flow: Student submits assignment via browser to web application

	Threat	Example
S	Submit as another student	Stolen session cookie, shared credentials
T	Submission modified in transit	MITM without TLS; post-deadline modification
R	"I never submitted that"	No logging of timestamps or content hashes
I	Other students view submission	Broken access control, IDOR on download URLs
D	Deadline flood crashes server	No rate limiting; malicious file upload
E	Student gains instructor role	Parameter tampering on role field

What Are We Going to Do About It?

Four response strategies for each identified threat:

1. **Mitigate** – Implement a control that reduces the risk (*most common*)
2. **Eliminate** – Remove the feature or component that creates the threat
3. **Transfer** – Shift risk to another party (insurance, outsourcing, user accepts ToS)
4. **Accept** – Acknowledge and *document* the decision not to act

From Threats to Security Requirements

Structure each requirement in three parts (SAFECode):

The Problem:

Unauthorized access to another student's submitted file
(CWE-639: IDOR)

The Control:

All file download requests must verify that the requesting user is authorized to access the specific file

Implementation Guidance:

Use indirect references (per-session token mapped to file path); validate enrollment and role on every request

Threat Modeling in the SDLC



The threat model's security requirements feed into:

- **Design review:** Are the controls designed correctly?
- **Test plan:** Are we testing for the threats we identified?
- **Bug bar:** Which threat-derived bugs block release?

Common Failures in Threat Modeling

Mindset failures (SAFECode Section 6):

- “We already do pen testing, so we don’t need threat modeling”
- “The system is already deployed, so there’s no reason”
- “We did it once; we don’t need to do it again”
- “Threat modeling is too complicated”
- “We don’t have security experts, so we can’t do it”

Process failures:

- Failing to control scope (“boiling the ocean”)
- Focusing only on areas the team already knows well
- Not defining what “success” looks like

Next Class: Hands-On Threat Modeling

We will threat model a peer-to-peer payment app (like Venmo/Zelle)

Think about:

- How do these apps work?
- What data do they handle?
- What could go wrong?

We'll drive the whole process together:

1. Draw the DFD
2. Apply STRIDE
3. Prioritize and develop mitigations

Further Reading

Assigned:

- CIS/SAFECode, *Secure by Design: A Guide to Assessing Software Security Practices* (2025)

Foundational:

- Saltzer & Schroeder, “The Protection of Information in Computer Systems” (1975)
- NIST SP 800-218, *Secure Software Development Framework*

Practical:

- Adam Shostack, *Threat Modeling: Designing for Security*
- Google, “Secure by Design at Google” (2024)
- CISA Secure by Design resources

See also: Appendix A of the paper for a comprehensive resource list.